Laboratorio Final: Galaxia en colisión

CC3069 · Computación Paralela y Distribuida · UVG · Ing. Juan Luis García Zarceño · Duración: 90 minutos

Por qué este laboratorio es distinto

Con Claude Code, escribir un kernel ya no demuestra que lo entendieron. Este laboratorio evalúa lo que la IA no puede hacer por ustedes: **predecir, medir, verificar y explicar**.

Van a simular **dos galaxias que chocan** (simulación N-body con gravedad) y producir una animación. Encima de eso:

- **Predicen antes de ejecutar.** Cada experimento empieza con una predicción escrita a mano; la nota depende del razonamiento, no de acertar.
- **Le ponen reglas a la IA.** Cada equipo escribe un `CLAUDE.md` con las restricciones de su proyecto.
- **Sabotaje galáctico.** Cada equipo esconde un bug de concurrencia, e intercambian máquinas con otro equipo para cazarlo **sin IA**.
- **Gran Premio.** Tabla de posiciones en vivo con el kernel más rápido que siga siendo correcto.

Conexión con la clase “El futuro es paralelo”

Cada fase pone a prueba, a pequeña escala, una tendencia vista en la última clase teórica.

Fase	Tendencia de la clase	Cómo aparece en el lab
1. Big Bang	La IA como el mayor programa paralelo	Un hilo por estrella = data parallelism, igual que una GPU por lote de datos.
2. Experimentos	Calcular es barato, mover datos no	Miden cuánto pesa `cudaMemcpy` y lo comparan con la suma de vectores.
3. Galaxias inteligentes	Menos bits y memoria cerca del cómputo	Tiling en memoria compartida y reto opcional en FP16.
4. Sabotaje	A escala, algo siempre falla (419 fallas en 54 días)	Diagnosticar un error sin saber dónde está.
5. Gran Premio	El límite es el enchufe	Además de velocidad, compiten en eficiencia energética.
Reflexión	Mismo colectivo, 30 años después	Qué colectivo de MPI o NCCL haría falta para usar 1,000 GPUs.

Datos generales

Modalidad: grupos de 3, una computadora con GPU NVIDIA y Claude Code por grupo. **Objetivo:** dirigir a una IA para construir una simulación paralela en CUDA y demostrar, con predicciones y mediciones propias, que entienden por qué es rápida, cuándo deja de escalar y cómo puede fallar.

Roles (rotan en la Fase 4)

Rol	Responsabilidad
Piloto	Maneja Claude Code: escribe los prompts, revisa cada diff antes de aceptarlo y ejecuta.
Científico	Escribe las predicciones antes de cada experimento, registra mediciones y arma las gráficas.
Auditor	Desconfía de la IA: compara GPU contra CPU, busca errores y debe poder explicar cualquier línea del kernel.

Reglas de uso de IA

1. Claude Code se puede usar para escribir y modificar código en las Fases 1, 2, 3 y 5.
2. **Toda predicción se escribe en la hoja de predicciones antes de ejecutar.** Una predicción escrita después vale cero.
3. En la **Fase 4 (Sabotaje)** el equipo cazador no puede usar IA hasta entregar su hipótesis escrita.
4. **Prueba de la línea:** en cualquier momento el catedrático puede señalar una línea del kernel y cualquier integrante debe explicarla. Si nadie puede, el equipo pierde 5 puntos.
5. Llevan una bitácora de prompts durante todo el lab y la entregan junto con el historial completo de Claude Code (`prompts.md`).

Bitácora de prompts (se evalúa)

El Piloto registra en `bitacora.md` cada prompt importante en el momento en que lo usa, no al final. La bitácora demuestra cómo dirigieron a la IA: qué le pidieron, qué contexto le dieron y qué hicieron con lo que devolvió.

#	Fase	Objetivo	Prompt (texto exacto)	Qué devolvió Claude	Decisión	Por qué
1	1	Kernel ingenuo de N-body	“…”	Kernel + versión CPU	Aceptado con cambios	No medía `cudaMemcpy` aparte, como exige el `CLAUDE.md`

Requisitos mínimos

- Al menos una entrada por fase (0 a 5).
- La columna **Decisión** usa: Aceptado, Aceptado con cambios o Rechazado.
- Al menos **2 casos** en los que corrigieron o rechazaron lo que propuso Claude, con una razón técnica.
- Al final, marquen su **prompt de oro**: el que mejor funcionó, y expliquen qué lo hizo bueno.
- La bitácora debe coincidir con el historial exportado; una bitácora reconstruida al final se nota.

Fases del laboratorio

Fase	Min	Qué hacen	Concepto
0. Despegue	5	Verificar GPU y escribir el `CLAUDE.md`	Especificar antes de programar
1. Big Bang	20	Simular dos galaxias que chocan, GPU contra CPU	Índices, un hilo por estrella
2. Experimentos	20	Medir tamaño de bloque, escalamiento y `cudaMemcpy`	Amdahl, memory wall
3. Galaxias inteligentes	15	Optimizar con memoria compartida por bloques (tiling)	`__shared__`, `__syncthreads()`
4. Sabotaje galáctico	15	Esconder un bug y cazar el del equipo vecino	Condiciones de carrera
5. Gran Premio	10	Benchmark de velocidad y de energía, y galería	Rendimiento por watt
Reflexión	5	Preguntas individuales	Metacognición

Fase 0: Despegue (5 min)

1. Ejecuten `nvidia-smi` y `nvcc --version`. Anoten el modelo de GPU. Descarguen el kit oficial (`kit_galaxia.zip`) y lean su README: su programa debe cumplir ese contrato de entrada y salida.
2. Creen la carpeta del proyecto y un `CLAUDE.md` con **al menos estas reglas**, más una inventada por el equipo:

- Todo cálculo de índice lleva un comentario en español explicando qué estrella o qué posición representa.
- Siempre debe existir una versión CPU de referencia, y cada ejecución imprime el error máximo GPU vs CPU.
- El tiempo del kernel y el de `cudaMemcpy` se miden por separado con `cudaEvent`.
- Prohibido usar Thrust, cuBLAS o cualquier librería que esconda el kernel.

Fase 1: Big Bang (20 min)

Construyan `galaxia.cu`: **N estrellas** repartidas en dos galaxias en forma de disco en curso de colisión. Cada paso de tiempo, cada estrella suma la atracción gravitacional de todas las demás ($O(N^2)$) y actualiza su posición.

- Un hilo por estrella. Kernel ingenuo, sin memoria compartida todavía.
- Versión CPU equivalente para comparar, y verificación contra la referencia del kit con N = 4,096 (`verificar_y_medir.py --n 4096`).
- Cada k pasos se guarda un frame como imagen (PPM o PNG) y al final se arma un GIF.

Predicción antes de ejecutar: con N = 4,096, ¿cuántas veces más rápida será la GPU que la CPU? ¿Qué porcentaje del tiempo total se irá en `cudaMemcpy`?

Preguntas de validación

1. Escriban la fórmula que convierte `blockIdx.x`, `blockDim.x` y `threadIdx.x` en el número de estrella. ¿Qué pasa con los hilos sobrantes si N no es múltiplo del tamaño de bloque?
2. ¿Por qué un hilo por estrella y no un hilo por par de estrellas? ¿Qué problema de concurrencia aparecería con un hilo por par?
3. ¿Por qué hay que sumar un pequeño valor de suavizado (softening) a la distancia?

Fase 2: Experimentos (20 min)

Cada experimento: **predicción escrita → medición → explicación de la diferencia.** Entreguen una tabla y una gráfica por experimento.

- **Experimento A, tamaño de bloque.** Con N = 16,384, midan el kernel con bloques de 32, 64, 128, 256, 512 y 1024 hilos. Predicción: ¿cuál será el más rápido?
- **Experimento B, escalamiento.** Midan CPU y GPU con N = 1,024, 4,096, 16,384 y 65,536 (la CPU puede detenerse si tarda más de 1 minuto). Predicción: si N se duplica, ¿cuánto crece el tiempo?
- **Experimento C, el muro de memoria.** Pídanle a Claude un kernel de suma de vectores con las mismas reglas de medición. Comparen qué porcentaje del tiempo se va en `cudaMemcpy` en la suma de vectores y en N-body. Es el muro de memoria de la clase de tendencias: traer un dato de DRAM cuesta cientos de veces más energía que operarlo.

Preguntas de validación

1. En la suma de vectores la GPU puede perder contra la CPU si se cuenta la copia de memoria; en N-body gana por mucho. ¿Por qué? (Pista: operaciones por dato transferido.)
2. Con el porcentaje de `cudaMemcpy` que midieron, apliquen la ley de Amdahl: ¿cuál es el speedup máximo posible si el kernel fuera infinitamente rápido?
3. ¿Por qué los bloques muy pequeños rinden peor? ¿Y por qué 1024 no siempre es el mejor?

Fase 3: Galaxias inteligentes (15 min)

Optimicen el kernel con **tiling en memoria compartida**: cada bloque carga un grupo de estrellas en `__shared__`, sincroniza, todos los hilos las usan, sincroniza y carga el siguiente grupo.

- Predicción: ¿cuánto más rápido será que el kernel ingenuo?
- El error máximo contra la versión ingenua debe ser menor a 10^{-3}.

Preguntas de validación (las responde el Auditor sin preguntarle a la IA)

1. El kernel tiene dos `__syncthreads()` dentro del ciclo. ¿Qué protege cada uno?
2. ¿Qué bug aparece si se quita el segundo? ¿Se vería siempre en la animación?
3. Si la memoria compartida es más rápida, ¿por qué no se guardan las N estrellas completas ahí?

Reto opcional: hacer matemáticas peor, a propósito. Si les sobra tiempo, pídanle a Claude una versión con `half` (FP16) para las posiciones. Antes de ejecutar, predigan cuánto cambia la velocidad y si el error sigue por debajo de 10^{-3}. Después respondan: ¿por qué a una red neuronal le afecta menos este error que a una simulación física?

Fase 4: Sabotaje galáctico (15 min)

En el entrenamiento de Llama 3 hubo 419 fallas inesperadas en 54 días: a gran escala, algo siempre está fallando. Esta fase entrena lo que hace un ingeniero de sistemas distribuidos: encontrar un error en un código que no escribió y que "casi" funciona.

Plantar (5 min). Copien su kernel a `sabotaje.cu` y pídanle a Claude que introduzca **un solo bug sutil**, por ejemplo: quitar un `__syncthreads()`, una condición de carrera en memoria compartida, un error de índice en el último tile o memoria compartida sin inicializar. Reglas: debe compilar, debe correr y la salida debe verse "casi" bien. Escriban la solución en un papel doblado y entréguenselo al catedrático.

Cazar (10 min). Intercambien computadora con el equipo asignado. Los roles rotan: el Auditor ahora es Piloto. Sin IA, entreguen una hipótesis escrita con tres partes: **línea, tipo de bug y por qué causa el síntoma que observan**. Después de entregarla pueden usar Claude para confirmar y corregir.

Fase 5: Gran Premio y galería (10 min)

- **Benchmark oficial:** N = 32,768, 10 pasos, mejor kernel del equipo. Métrica: **interacciones por segundo** = N^2 × pasos ÷ tiempo del kernel. Solo cuenta si el error contra la referencia es menor a 10^{-3}. Se mide con el kit oficial, igual para todos: `python3 verificar_y_medir.py ./galaxia --equipo "Nombre"`.
- Cada equipo anota su resultado en la tabla del pizarrón.
- Se proyectan los GIFs y el salón vota la colisión más bonita.

Gran Premio Verde. “El próximo límite no es el transistor, es el enchufe.” Corran el benchmark agregando `--energia` al script del kit: mide la potencia con `nvidia-smi` mientras corre 100 pasos. Métrica: **interacciones por joule** = N^2 × pasos ÷ (potencia promedio en W × tiempo del kernel en s). Gana el kernel más eficiente, no necesariamente el más rápido.

Reflexión individual (5 min)

1. ¿Qué predicción falló por más y qué no habías considerado?
2. Describe algo que Claude hizo mal o de forma subóptima y cómo lo detectaron.
3. Explica a alguien de primer año, con una analogía, por qué la versión con tiling es más rápida.
4. Si repartieran las estrellas entre 1,000 GPUs, cada GPU necesitaría las posiciones de todas las demás en cada paso. ¿Qué operación colectiva de MPI (o NCCL) usarían y qué limitaría el speedup según Amdahl?
5. ¿Qué parte de este lab no delegarías a una IA, y por qué?

Entregables

Cada equipo sube un `.zip` al final de la clase con:

1. `CLAUDE.md`, `galaxia.cu` (ingenuo y con tiling) y `sabotaje.cu`.
2. `prompts.md` con el historial completo exportado de Claude Code, y `bitacora.md` con la bitácora de prompts.
3. La hoja de predicciones y las tablas y gráficas de la Fase 2.
4. El GIF de la colisión.

5. El reporte de sabotaje: la hipótesis escrita del bug que cazaron y la corrección.
6. Respuestas a las preguntas de validación y la reflexión individual de cada integrante.

Evaluación

Criterio	Puntos
Predicciones y experimentos	25
Correctitud y verificación	15
Preguntas de validación	15
Bitácora de prompts	10
Sabotaje	15
Reflexión individual	15
Gran Premio	5 (+2 colisión más votada, +2 Gran Premio Verde)
Prueba de la línea	−5 por cada línea sin explicar

La rúbrica detallada por niveles se publica junto con este enunciado.

Hoja de predicciones

Equipo: _____ Piloto: _____ Científico: _____ Auditor: _____

Llenen la columna de predicción **antes** de ejecutar. El catedrático puede firmar la columna de predicciones al pasar por su mesa.

#	Fase	Pregunta	Predicción (antes)	Medido	¿Por qué la diferencia?
1	1	Speedup GPU vs CPU con N = 4,096			
2	1	% del tiempo total en cudaMemcpy			
3	2A	Tamaño de bloque más rápido			
4	2B	Si N se duplica, el tiempo se multiplica por…			
5	2C	% en cudaMemcpy: suma de vectores vs N-body			
6	2C	Speedup máximo según Amdahl			
7	3	Speedup del tiling vs kernel ingenuo			
8	5	Interacciones por segundo en el Gran Premio			
9	5	Interacciones por joule en el Gran Premio Verde			